Project: Iris Species Identification

Dataset Used: Iris.csv

This document outlines a structured, complete report of project based on training the module, predicting the test data and evaluating the accuracy on given test data.

Project Goal: Train a module to predict the classes using Iris.csv dataset.

The primary libraries/modules required for this project are:

- **Pandas**: For handling datasets.
- **NumPy**: For numerical operations.
- **sklearn**: For statistical machine learning.

Project Coverage:

Category	Skills Covered
Data Pre-Processing	Data Loading.
Create Learning Dataset	Select the required columns for passing as Input.
Training the module	Algorithms to train the module using the dataset.
Predict the result	Algorithms to predict the species class

In the code we are importing the required libraries to load the dataset and creating a training set to train the module. After training using the dataset, we are using different classes from the sklearn module to predict the species class with different algorithms as shown below.

1. Algorithm: Support Vector Machine (SVM)

- SVM tries to find the best boundary (line/plane) to separate classes.
- It focuses on maximizing the margin between different classes.

Once the training and testing datasets are created we can use those to test and predict the accuracy using the code as shown below.

```
from sklearn.svm import SVC
classifier = SVC()
print(classifier)

# Train model
classifier.fit(X_train, y_train)

# Predict test data
y_pred = classifier.predict(X_test)

# Evaluate accuracy
from sklearn.metrics import accuracy_score
print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

In this algorithm we are importing SVC class from svm module and training the module with the training set. After that we are predicting the accuracy and printing it.

2. Logistic Regression

- Logistic Regression predicts probability and classifies using a threshold.
- It works well for linearly separable data.

```
from sklearn.linear_model import LogisticRegression
classifier = LogisticRegression()
print(classifier)

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

In this algorithm we are importing LogisticRegression classs from linear_model sub-module and using fit function to train the module. After that predicting the accuracy and printing it.

3. Naive Bayes

- Naive Bayes assumes features are independent of each other.
- It uses probability to predict the most likely class.

```
from sklearn.naive_bayes import GaussianNB
classifier = GaussianNB()
print(classifier)

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

In this algorithm we are importing GaussianNB class from naive_bayes sub-module and using fit function to train the module. After that predicting the accuracy and printing it.

4. Decision Tree Classifier

- Decision Tree splits data into branches based on feature values.
- It makes decisions like a flowchart.

```
from sklearn.tree import DecisionTreeClassifier
classifier = DecisionTreeClassifier()
print(classifier)

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

In this algorithm we are importing DecisionTreeClassifier class from tree sub-module and using fit function to train the module. After that predicting the accuracy and printing it.

5. Random Forest Classifier

- Random Forest builds multiple decision trees and combines their results.
- It improves accuracy and reduces overfitting.

```
from sklearn.ensemble import RandomForestClassifier
classifier = RandomForestClassifier()
print(classifier)

classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print('\n'+'-'*20+'Accuracy Score on the Test set'+'-'*20)
print("{:.0%}".format(accuracy_score(y_test, y_pred)))
```

In this algorithm we are importing RandomForestClassifier class from ensemble sub-module and using fit function to train the module. After that predicting the accuracy and printing it.

What you have learned:

In this project we learned the following.

- ➔ How to read the dataset using pandas.
- ➔ How to perform numerical operation on the dataset.
- ➔ How to create and Train the Linear Regression Model.
- ➔ How to test the Linear Regression model.
- ➔ How predict the accuracy with different algorithms.

Conclusion:

In this project we load the dataset to pandas library and created the training module for the same. After that we test the model and created the accuracy percentage using different algorithms and compared the results in the output.